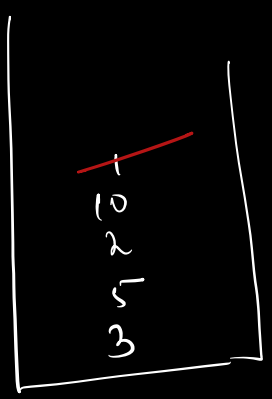
Q 1. Implement a Stack with the following functions

TC O(1)

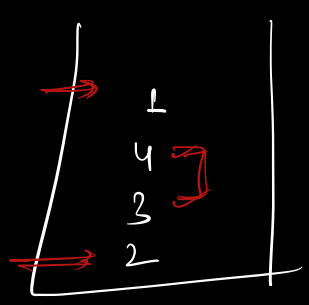
1) push()
2) pop()
3) getMin() → return the minm currently in stack

ex →



minm → ~~3~~ 2 1

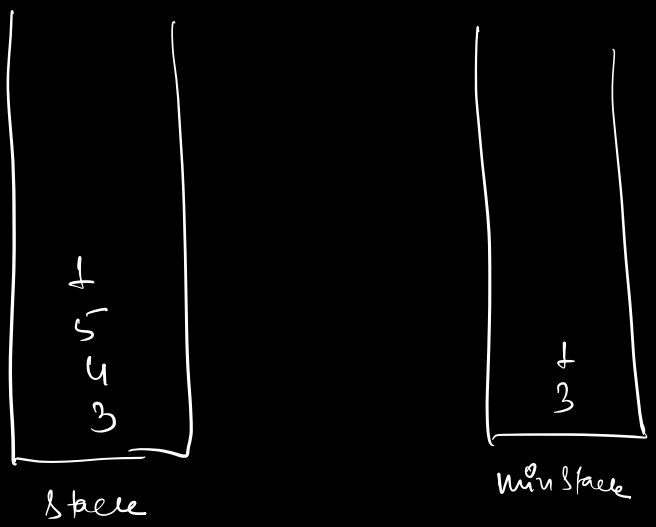
push(3) ←
push(5) ←
getMin() → 3 ←
push(2) ←
getMin() → 2 ←
push(10) ←
push(1) ←
getMin() → 1 ←
pop()
getMin() → 2.



minm → ~~2~~ (1)

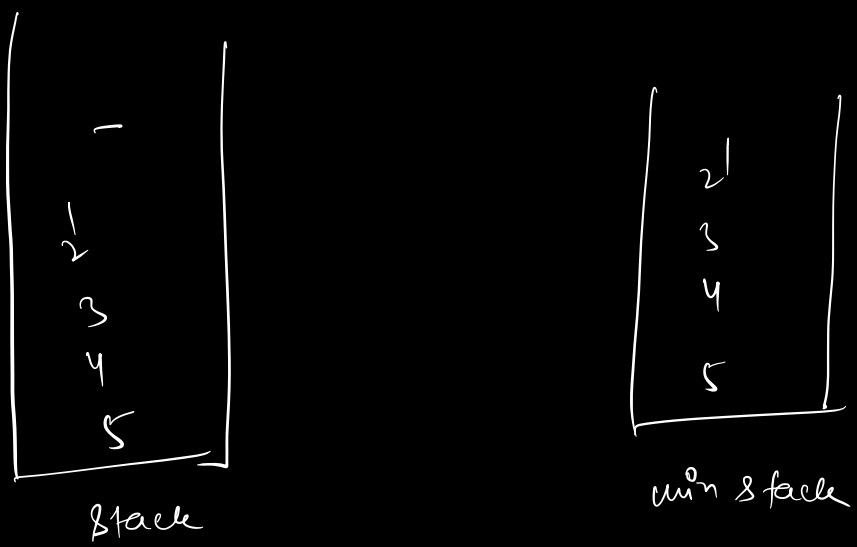
→ push(2)
→ push(7)
→ push(4)
→ push(1)
pop()

* maintain two stacks
  → 1) stack for actual values
  2) stack for mins [minstack].



⇒ pushing a new min → top of min stack.
(element ≤ curr min)
push into minstack as well

⇒ popping → if popped element == top of minstack
pop from both.



st → stack
minst → min stack

pseudo

```
push(a) {
    st.push(a)
    if( a <= minst.top() ) {
        minst.push(a)
    }
}
```

→ O(1)

```
pop() {
    int element = st.pop()
    if( element == minst.top() ) {
        minst.pop()
    }
    return element;
}

getMin() { return minst.top(). }
```

→ O(1)

→ O(1)

SC :: O(N)

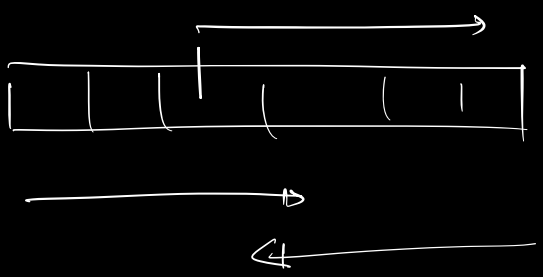
Intro to Trees

⇒ DS till now [Linear DS]

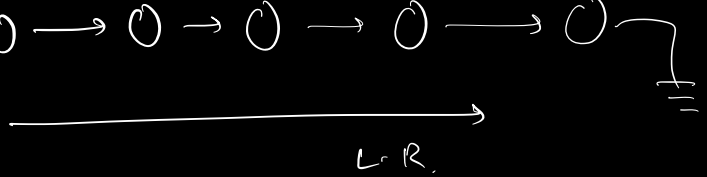
- → arrays
- → LL
- → Stacks
- → Queues

⇒ Iterations :

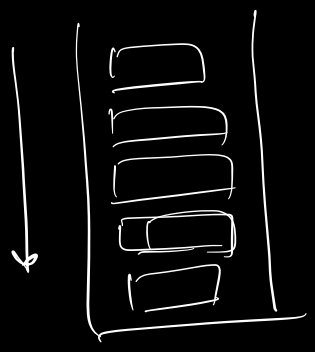
1) array



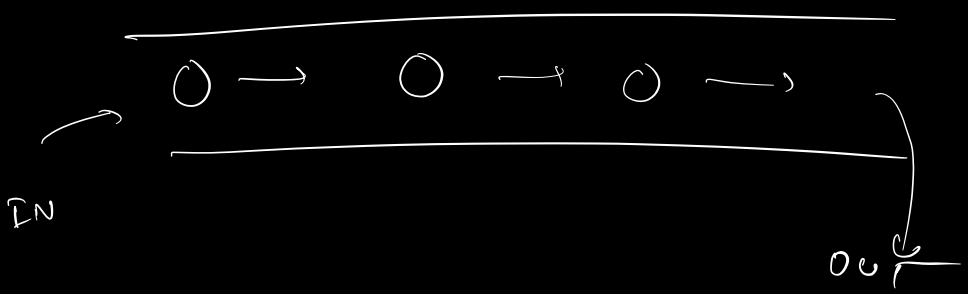
2) LL

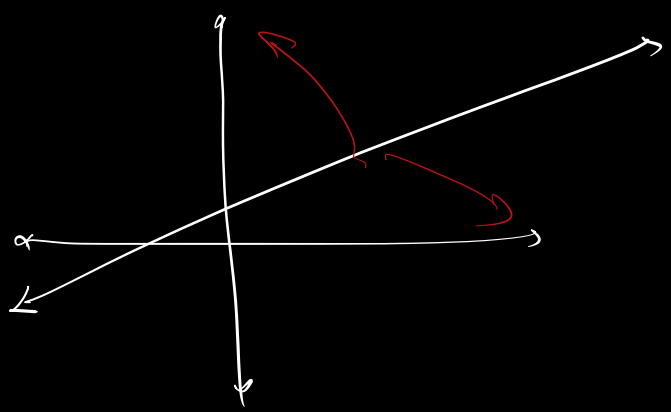


III) Stack



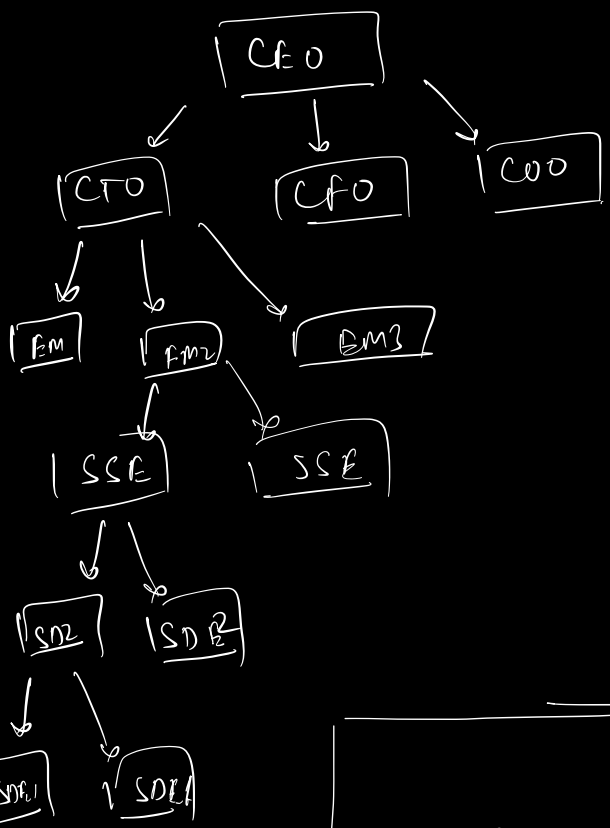
IV) Queue

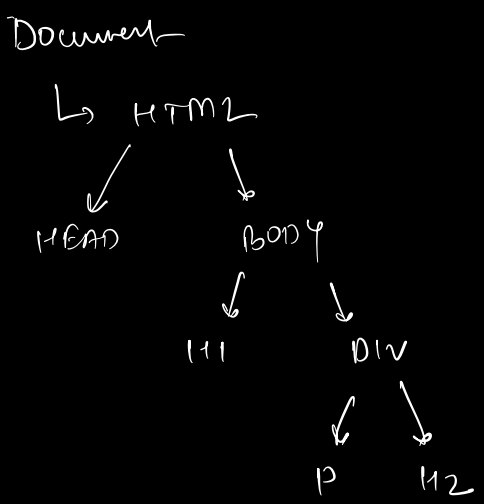




Tree → Non-linear

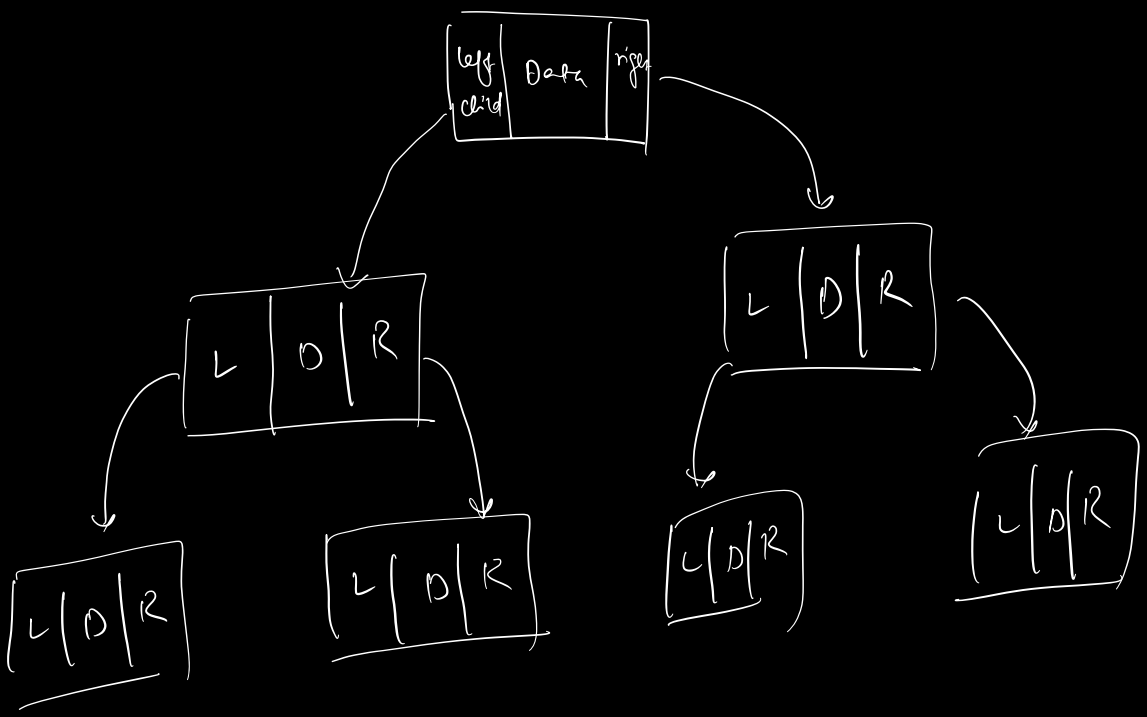
→ Hierarchical

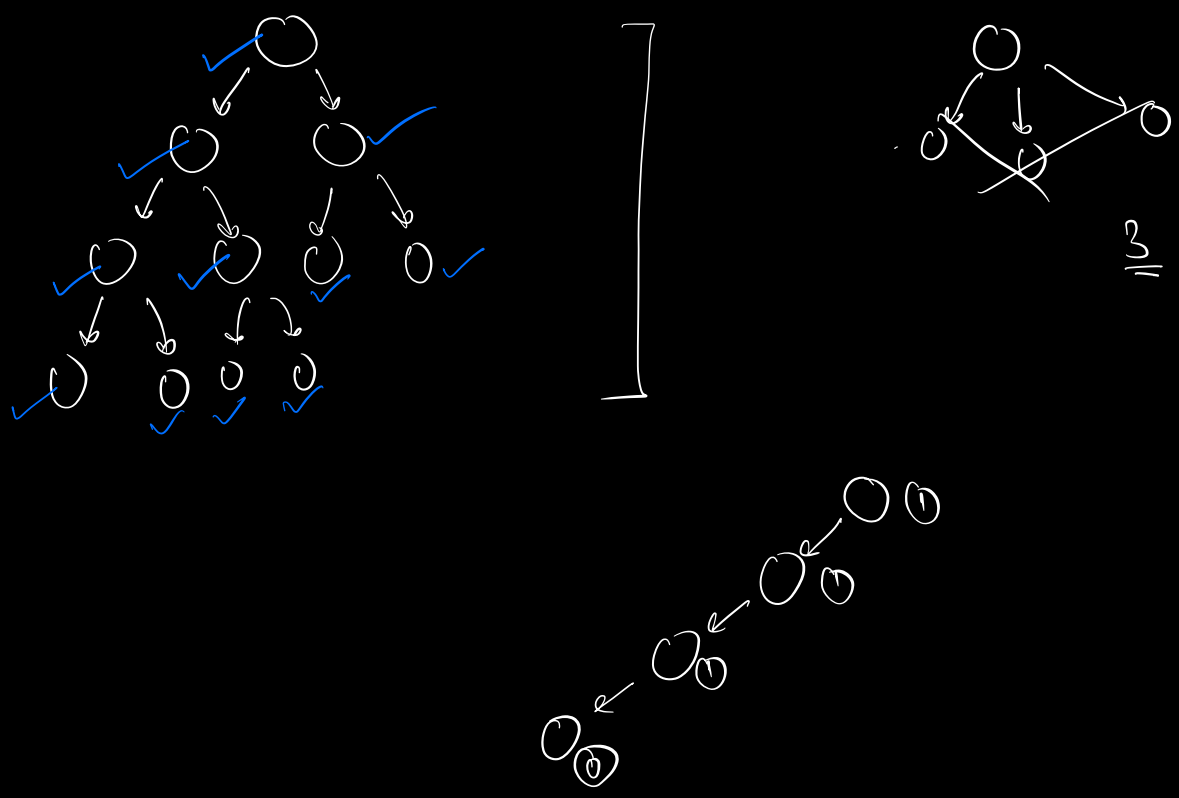




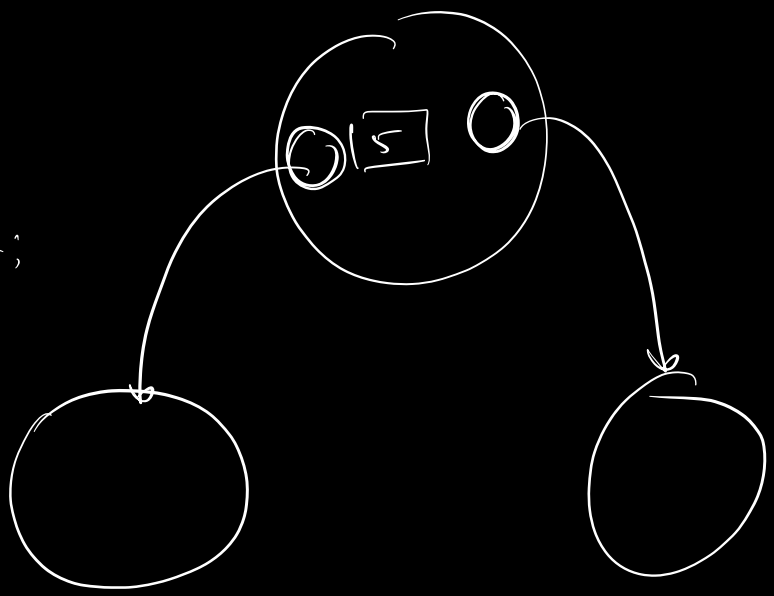
Binary Trees

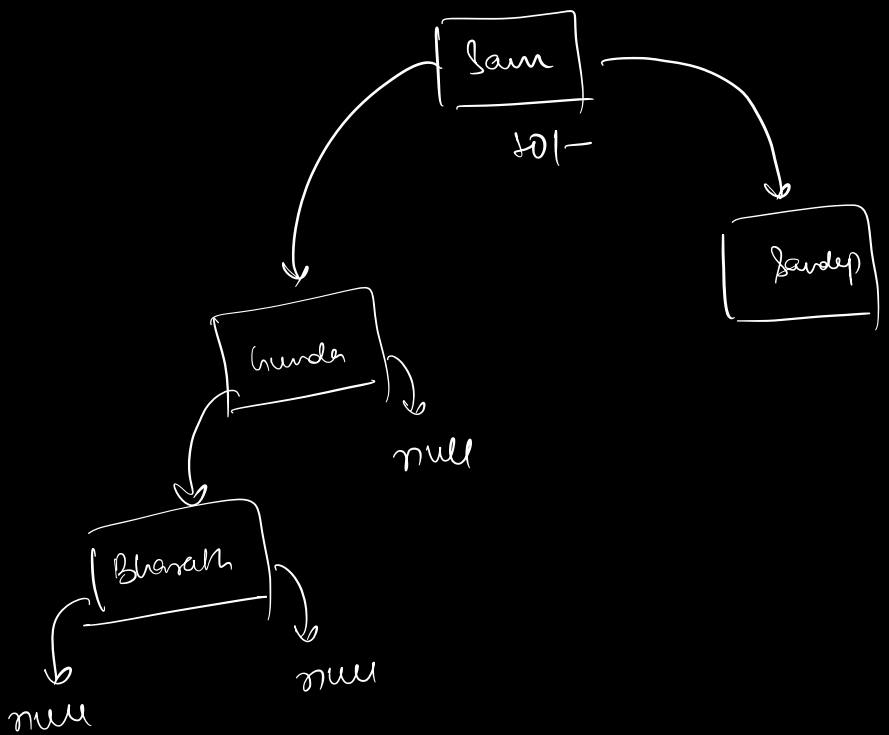
each node [0-2] children


left child
Data
right
L
D
R

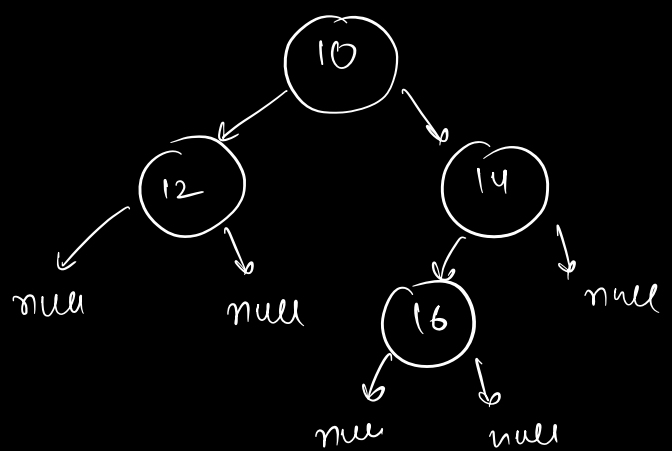

3

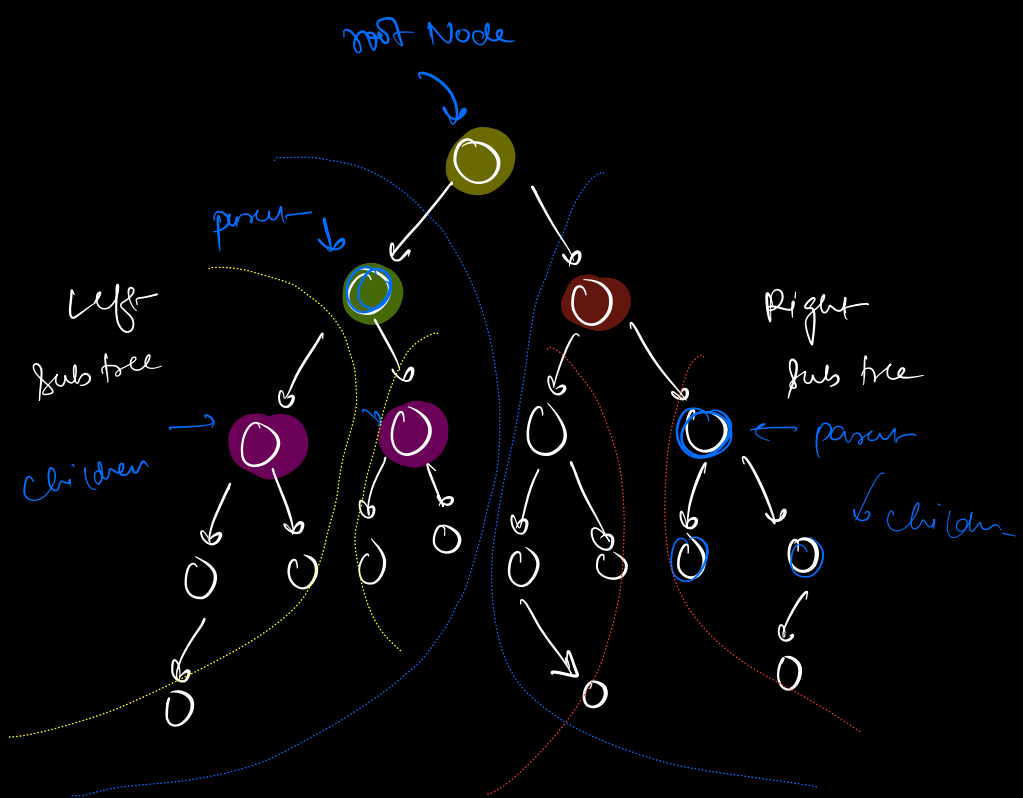
```
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;
}

public TreeNode(x) {
    data = x
    left = null
    right = null
}
```

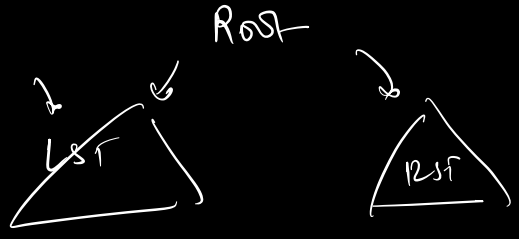


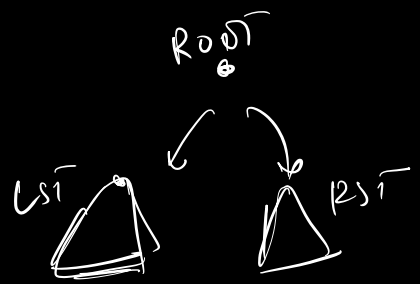


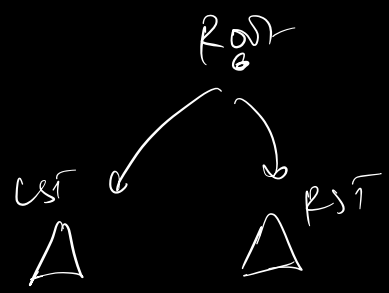
```
TreeNode t = new TreeNode(10);
t.left = new TreeNode(12);
t.right = new TreeNode(14);
t.right.left = new TreeNode(16);
```

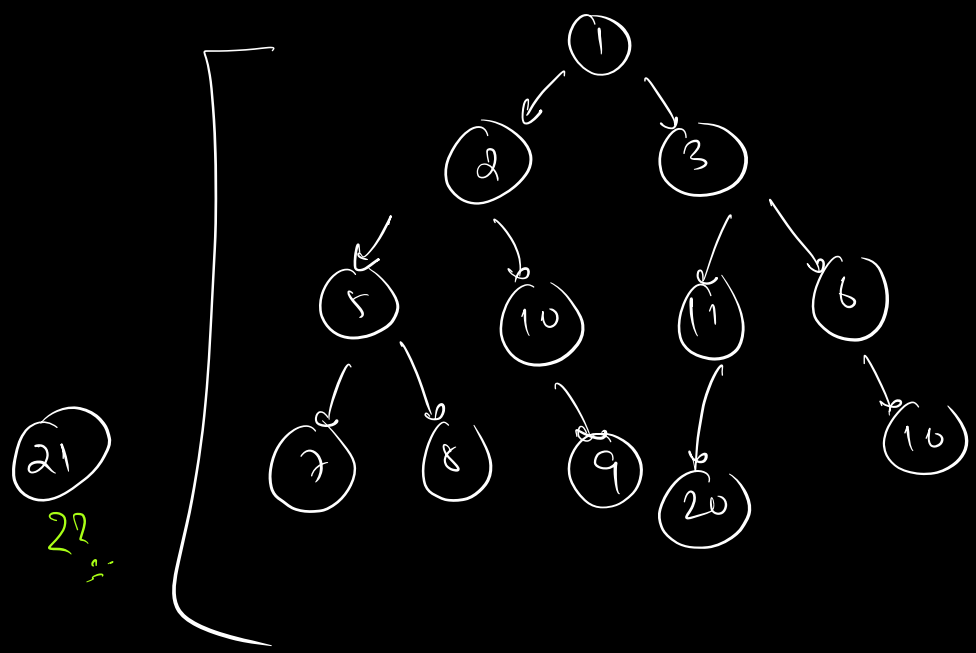





Root
LST
RST


Root
LST
RST


Root
LST
RST


1
2
3
5
10
11
6
21
7
8
9
20
16
22

1) root / parent

2) left

3) right

1) root
2) left
3) right

Pre order

1) left
2) root
3) right

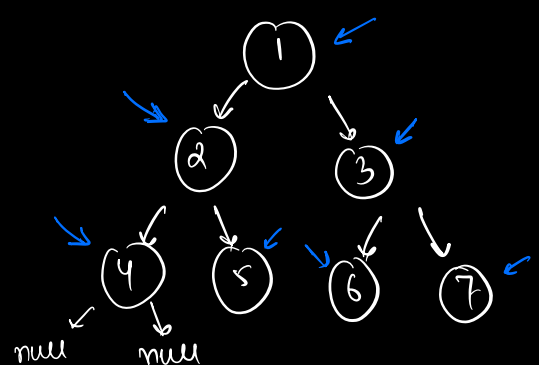
In order

1) left
2) right
3) root

Post Order

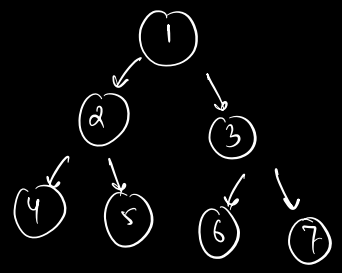
pseudo

1) preorder [root → left → right]

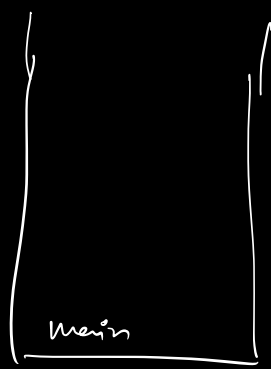


1 2 4 5 3 6 7 preorder traversal

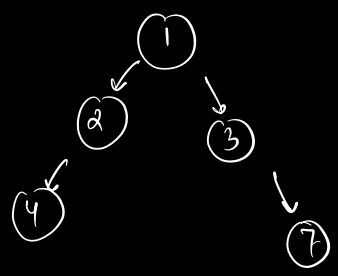
```
void preorder(TreeNode root){
    if(root == null)
        return;
    print(root.data)
    preOrder(root.left)
    preorder(root.right)
}
```



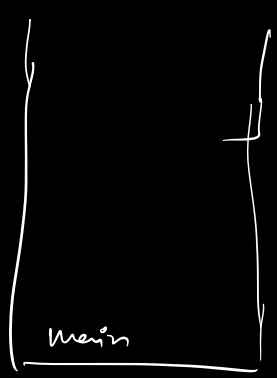
1 2 4 5 3 6 7 → pre Order



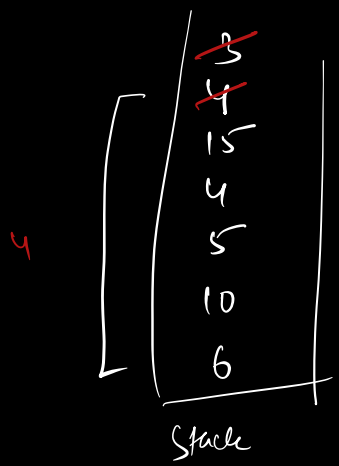

```
void preorder(TreeNode root){
    if(root == null)
        return; ←
    print(root.data)
    preOrder(root.left)
    preorder(root.right)
}
```

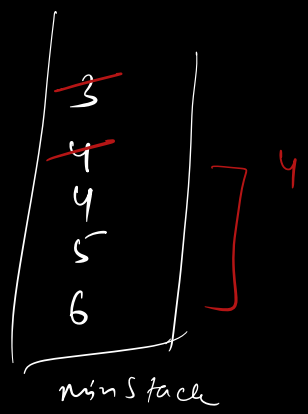


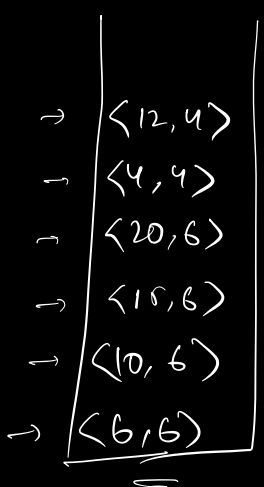
1 2 4 3 7 →

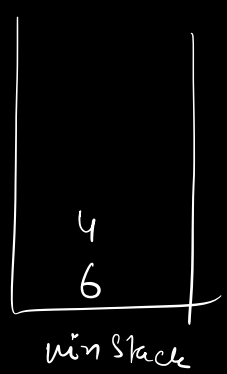


Doubts


5
4
15
4
5
10
6
4
Stack


3
4
4
5
6
4
min stack


→ (12,4)
→ (4,4)
→ (20,6)
→ (15,6)
→ (10,6)
→ (6,6)


4
6
min stack